

PYTHON PROJECT

VIDEO LINK:

[HTTPS://DRIVE.GOOGLE.COM/FILE/D/14XOXXYPHME7LOLILX9LOCLKNXDCLSGWZ/VIEW?USP=SHARING](https://drive.google.com/file/d/14XOXXYPHME7LOLILX9LOCLKNXDCLSGWZ/view?usp=sharing)

TASK I

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from scipy import stats
```

```
[2]: df = pd.read_csv("jp_morgan.csv")
df.head()
```

Explanation:

Imported necessary python libraries to perform analysis and imported stats for statistical tests.

Reads the csv file into a data frame table and shows first 5 rows.

```
[4]: df.info()
```

-: Shows columns names, data types, and missing values.

```
[5]: df.describe()
```

-: Gives statistical summary.

```
[6]: df.isnull().sum()
```

-: Counts missing values in each column.

```
[7]: df['TransactionDate'] = pd.to_datetime(df['TransactionDate'],dayfirst=True)  
df = df.sort_values(by='TransactionDate')
```

-: Converts strings into proper date format and sorts in chronological order.

```
[8]: df['TransactionType'] = df['TransactionType'].str.strip()  
df['AccountType'] = df['AccountType'].str.strip()
```

-: Removes extra spaces from columns.

TASK 2

```
[11]: df['Month'] = df['TransactionDate'].dt.to_period('M')

monthly_summary = df.groupby(['Month', 'TransactionType'])['TransactionAmount'].sum().unstack()
monthly_summary['Net'] = monthly_summary['Deposit'] - monthly_summary['Withdrawal']
monthly_summary.head()
```

-: Monthly summaries of total credits, debits and net transactions.

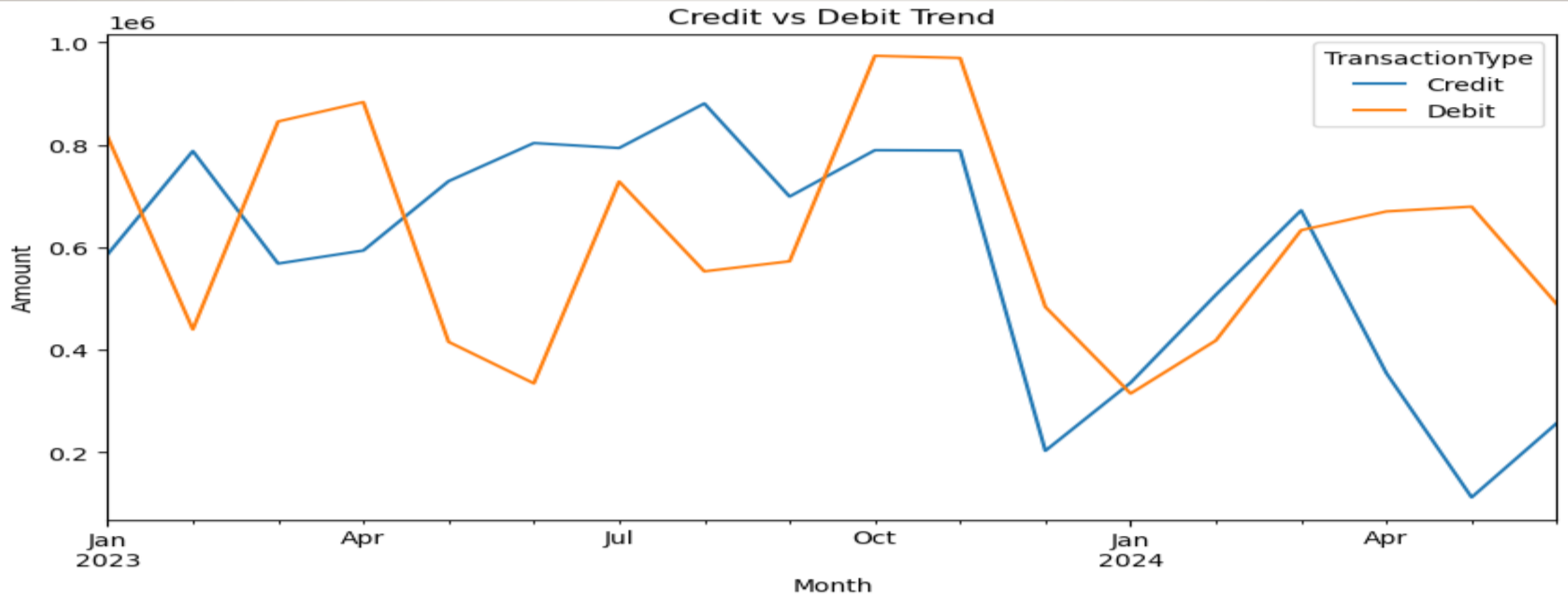
```
[13]: df['Year'] = df['TransactionDate'].dt.year

yearly_summary = df.groupby(['Year', 'TransactionType'])['TransactionAmount'].sum().unstack()
yearly_summary['Net'] = yearly_summary['Deposit'] - yearly_summary['Withdrawal']
yearly_summary.head()
```

-: Yearly summaries of total credits, debits and net transactions.

```
[14]: monthly_summary[['Deposit', 'Withdrawal']].rename(columns={
        'Deposit': 'Credit',
        'Withdrawal': 'Debit'
    }).plot(figsize=(10,5))
plt.title("Credit vs Debit Trend")
plt.xlabel("Month")
plt.ylabel("Amount")
plt.show()
```

Trends in total credits vs. debits over time.



```
account_flow = df.groupby('AccountID')['TransactionAmount'].sum()

top_accounts = account_flow.sort_values(ascending=False).head(10)
bottom_accounts = account_flow.sort_values().head(10)
```

Top and bottom performing accounts based on net inflow

top_accounts

```
[16]: AccountID
ACC42710    539494.876440
ACC45907    532556.635574
ACC81631    483854.558670
ACC32627    471468.023823
ACC18140    458219.210460
ACC95774    453488.129360
ACC55331    448188.410346
ACC29477    443067.484857
ACC29646    439477.177961
ACC22036    437626.170698
Name: TransactionAmount, dtype: float64
```

[17]: bottom_accounts

```
[17]: AccountID
ACC12334    15159.688345
ACC50439    24088.157284
ACC65144    30425.901148
ACC28305    32162.348245
ACC43771    32819.585640
ACC49180    37023.532950
ACC46655    37365.268690
ACC30146    42545.848430
ACC39482    42831.484830
ACC87006    42903.165440
Name: TransactionAmount, dtype: float64
```

```
[19]: df = df.sort_values(['AccountID', 'TransactionDate'])

df['Prev_Date'] = df.groupby('AccountID')['TransactionDate'].shift(1)
df['Gap'] = (df['TransactionDate'] - df['Prev_Date']).dt.days

dormant_accounts = df[df['Gap'] > 60]['AccountID'].unique()
```

List of Dormant accounts

```
array(['ACC10996', 'ACC11062', 'ACC11188', 'ACC11285', 'ACC12182',
       'ACC12334', 'ACC13357', 'ACC15228', 'ACC15359', 'ACC15671',
       'ACC15925', 'ACC16664', 'ACC17688', 'ACC18140', 'ACC19156',
       'ACC19178', 'ACC20297', 'ACC21264', 'ACC21429', 'ACC21719',
       'ACC21878', 'ACC22036', 'ACC22255', 'ACC22799', 'ACC23736',
       'ACC23985', 'ACC24070', 'ACC24508', 'ACC24880', 'ACC24981',
       'ACC25132', 'ACC25811', 'ACC26026', 'ACC26956', 'ACC26973',
       'ACC28154', 'ACC28295', 'ACC28305', 'ACC28612', 'ACC29007',
       'ACC29231', 'ACC29356', 'ACC29396', 'ACC29477', 'ACC30787',
       'ACC30852', 'ACC31539', 'ACC31902', 'ACC32212', 'ACC32627',
       'ACC32890', 'ACC33287', 'ACC34119', 'ACC34431', 'ACC34568',
       'ACC34821', 'ACC35163', 'ACC35419', 'ACC36079', 'ACC37688',
       'ACC38559', 'ACC39161', 'ACC39500', 'ACC39529', 'ACC39544',
       'ACC40952', 'ACC42467', 'ACC42710', 'ACC42903', 'ACC43309',
       'ACC43771', 'ACC45101', 'ACC45907', 'ACC45951', 'ACC45968',
       'ACC46655', 'ACC47099', 'ACC48303', 'ACC48501', 'ACC49140',
       'ACC49395', 'ACC49422', 'ACC50439', 'ACC50817', 'ACC51009',
       'ACC51200', 'ACC51593', 'ACC51971', 'ACC52131', 'ACC52650',
       'ACC53466', 'ACC53865', 'ACC54589', 'ACC55331', 'ACC55729',
       'ACC57516', 'ACC57597', 'ACC57700', 'ACC57872', 'ACC58078',
       'ACC58667', 'ACC60432', 'ACC61827', 'ACC61926', 'ACC62446',
       'ACC64393', 'ACC64785', 'ACC65144', 'ACC65545', 'ACC66086',
       'ACC67701', 'ACC67713', 'ACC69323', 'ACC70314', 'ACC70460',
       'ACC70741', 'ACC71388', 'ACC71426', 'ACC71938', 'ACC72197',
       'ACC73104', 'ACC74631', 'ACC74656', 'ACC75767', 'ACC76699',
       'ACC77533', 'ACC77592', 'ACC77638', 'ACC77773', 'ACC78178',
       'ACC78581', 'ACC78589', 'ACC80131', 'ACC80442', 'ACC81631',
       'ACC82298', 'ACC82381', 'ACC82926', 'ACC83269', 'ACC83581',
       'ACC83848', 'ACC86784', 'ACC87602', 'ACC88074', 'ACC88252',
```

TASK 3

```
[21]: freq = df.groupby('AccountID').size()

[22]: # High: > 8 transactions
      # Medium: 4-8
      # Low: < 4

[23]: def activity_level(x):
      if x > 8:
          return "High"
      elif x >= 4:
          return "Medium"
      else:
          return "Low"

      activity = freq.apply(activity_level)
```

Explanation: First I created freq variable which counts no. of transactions per account. Then created a custom function activity level where I ran a loop grouping them into High, Medium and Low based on there frequency and applied the function to each account.

```
[25]: profile = df.groupby('AccountID').agg({
        'AccountBalance': 'mean',
        'TransactionAmount': 'sum'
    }).rename(columns={
        'AccountBalance': 'Avg_Balance',
        'TransactionAmount': 'Total_Volume'
    })
```

-: Here I created profile variable where I segmented customers by there avg. balance and total transaction.

```
[28]: net_flow = df.groupby('AccountID').apply(
        lambda x: x[x['TransactionType']=='Deposit']['TransactionAmount'].sum() -
        x[x['TransactionType']=='Withdrawal']['TransactionAmount'].sum(),include_groups=False
    )

    high_inflow = net_flow[net_flow > net_flow.quantile(0.75)]
```

-: Here I created net flow variable in which Lambda is used to perform a custom calculation on each account group, where I compute net transaction flow by subtracting total debits from total credits.

```
[30]: high_freq_low_balance = profile[
        (profile['Avg_Balance'] < profile['Avg_Balance'].quantile(0.25)) &
        (freq > freq.quantile(0.75))
    ]
```

High inflow : accounts having net flow greater than net flow of 75% of accounts.

High freq low balance : accounts having frequency of greater than 75% and avg. balance is less than avg. balance of bottom 25% accounts.

```
[32]: risky_balance = profile[profile['Avg_Balance'] <= 0]
```

Risky balance : accounts having avg. balance less than or equal to 0.

TASK 4

```
[35]: large_withdrawals = df[
      (df['TransactionType'] == 'Withdrawal') &
      (df['TransactionAmount'] > df['TransactionAmount'].quantile(0.95))
      ]
      large_withdrawals
```

-: Extracted the top 5% large withdrawals

```
[37]: volatility = df.groupby('AccountID')['AccountBalance'].std()
```

-: Calculated balance volatility using standard deviation

```
[39]: Q1 = df['TransactionAmount'].quantile(0.25)
      Q3 = df['TransactionAmount'].quantile(0.75)
      IQR = Q3 - Q1

      outliers = df[
          (df['TransactionAmount'] < Q1 - 1.5*IQR) |
          (df['TransactionAmount'] > Q3 + 1.5*IQR)
      ]
```

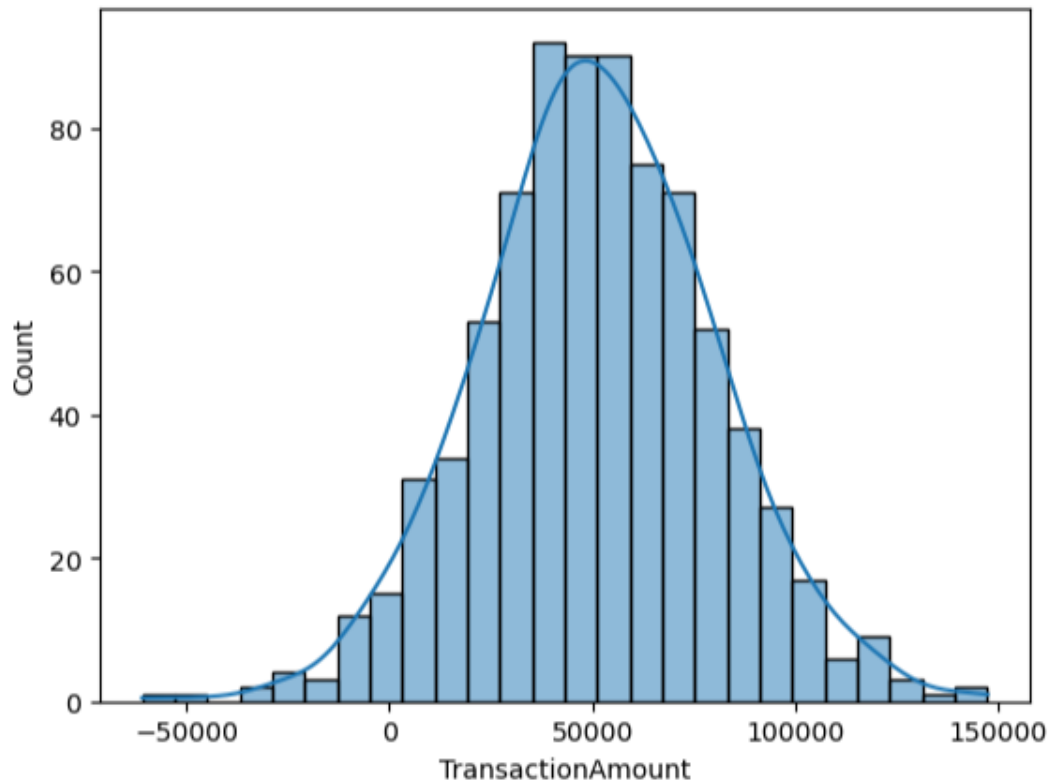
-: Used IQR method to detect outliers

[39]:	TransactionID	CustomerID	AccountID	AccountType	TransactionType	Product	Firm	Region	Manager	TransactionDate	TransactionAmount	AccountBalance	R
97	67	CUST6391	ACC26940	Current	Payment	Home Loan	Firm A	South	Manager 4	2023-03-16	-25964.94612	94822.29734	
129	161	CUST4373	ACC41829	Current	Payment	Savings Account	Firm A	North	Manager 3	2024-03-22	135972.34390	42302.00951	-
701	191	CUST6837	ACC46655	Current	Deposit	Mutual Fund	Firm E	West	Manager 2	2024-06-16	-29563.97803	49810.32678	
53	178	CUST1189	ACC50439	Savings	Deposit	Savings Account	Firm D	East	Manager 1	2024-06-16	-60979.07348	57916.06101	
659	21	CUST1644	ACC77533	Credit	Payment	Personal Loan	Firm D	South	Manager 4	2023-12-24	-30826.73980	68006.03943	
397	51	CUST1497	ACC88074	Current	Transfer	Mutual Fund	Firm B	East	Manager 3	2024-03-05	-45352.95439	96125.56424	-
727	163	CUST1747	ACC92104	Savings	Deposit	Savings Account	Firm A	West	Manager 1	2023-08-06	147447.29510	101759.29850	
242	86	CUST7855	ACC95164	Savings	Payment	Home Loan	Firm C	North	Manager 3	2023-10-26	143067.18460	73627.79044	
579	50	CUST1749	ACC99117	Current	Payment	Personal Loan	Firm E	West	Manager 4	2023-11-29	-24819.84559	101124.98330	
115	122	CUST8344	ACC99117	Credit	Withdrawal	Home Loan	Firm B	East	Manager 4	2024-01-29	129471.46560	73117.85560	

-: outlier accounts

TASK 5

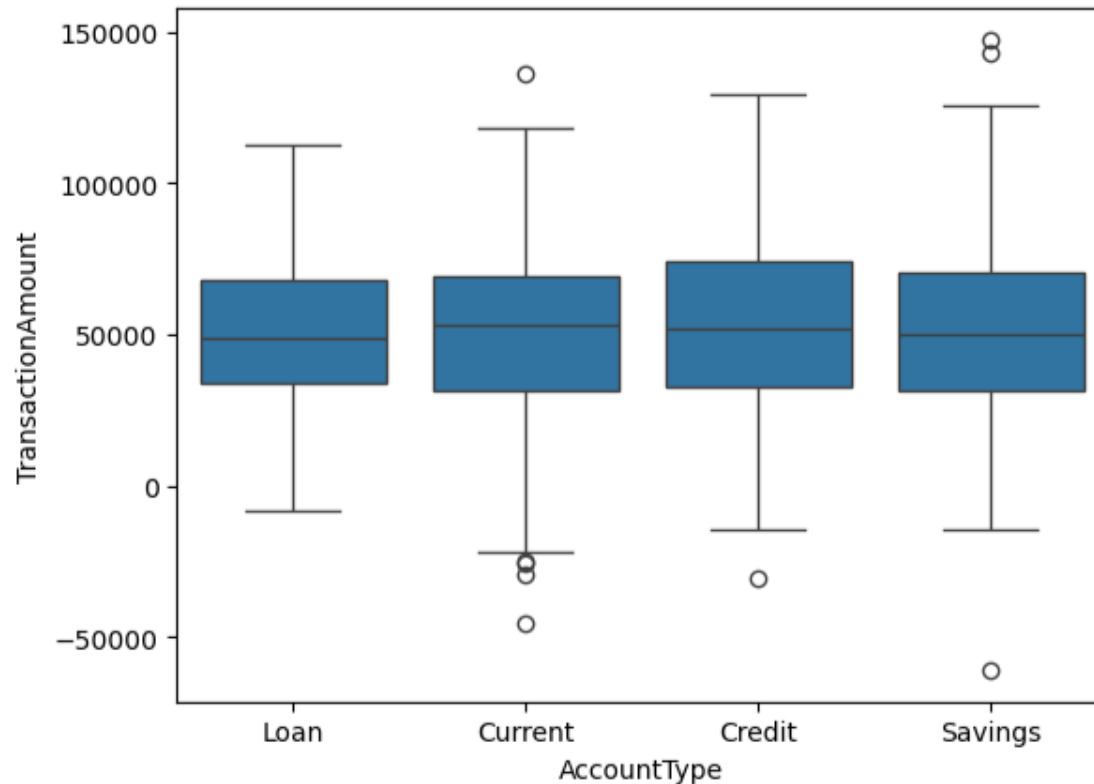
```
[42]: sns.histplot(df['TransactionAmount'], kde=True)  
plt.show()
```



Explanation: This histogram shows the distribution of transaction amounts across all accounts.

As we can see from the distribution of Transaction Amounts, the data follows a near-perfect normal distribution. With a peak frequency at 50,000, we can confidently identify our core transaction bracket. The lack of heavy skewness suggests that there aren't many extreme 'whale' transactions or unusually tiny payments pulling the average away from the center.

```
[44]: sns.boxplot(x='AccountType', y='TransactionAmount', data=df)
plt.show()
```



Explanation: This boxplot compares transaction amounts across different account types.

When we segment the transaction data by account type, the most striking takeaway is the uniformity. Whether it is a Loan, Credit, Current, or Savings account, the median transaction remains steady at approximately 50,000. This suggests a universal transaction behavior across our product lines, with Savings and Current accounts being the primary drivers of high-value outlier activity.

```
[46]: corr = df[['TransactionAmount', 'AccountBalance']].corr()  
sns.heatmap(corr, annot=True)  
plt.show()
```



Explanation: This heatmap shows the correlation between transaction amount and account balance. Interestingly, our heatmap reveals that there is virtually zero correlation specifically -0.03 between how much money a customer keeps in their account and the size of their individual transactions. This tells us that transaction behavior is driven by factors other than wealth or liquidity.

TASK 6

```
[49]: threshold = profile['Total_Volume'].median()

high_volume = profile[profile['Total_Volume'] > threshold]['Avg_Balance']
low_volume = profile[profile['Total_Volume'] <= threshold]['Avg_Balance']
```

-: This code calculates a median threshold for Total Volume to split the dataset into two balanced segments: High Volume and Low Volume. It then extracts the Avg Balance for each group.

```
[52]: t_stat, p_value = stats.ttest_ind(high_volume, low_volume)

print("T-stat:", t_stat)
print("P-value:", p_value)

T-stat: -0.4331698271374088
P-value: 0.6653776293187159

[53]: if p_value < 0.05:
        print("Reject H0 → High-volume accounts have higher balances")
    else:
        print("Fail to reject H0")

Fail to reject H0
```

-: I performed a t-test to see if high-volume accounts hold larger balances, but with a p-value of 0.665, we failed to find a significant link. Since this value is well above the 0.05 threshold, we conclude that any difference in balances is likely due to random chance.